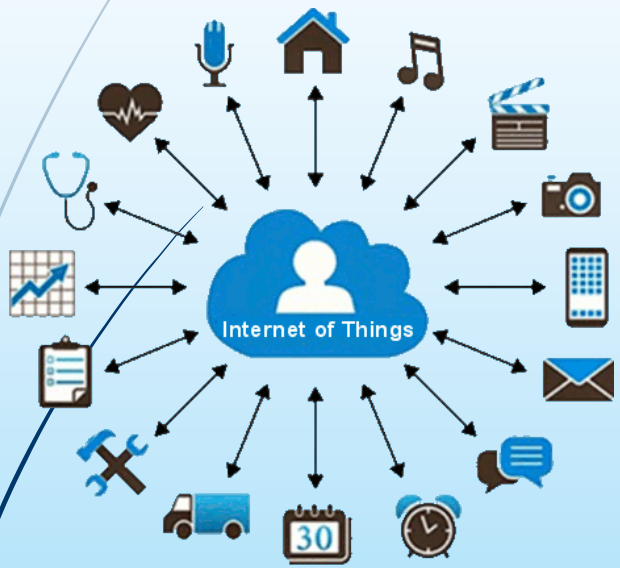
EPC for IoT - Zephyr OS



- 4th Week: Hands-on-lab
  - Exercise specification

Hands-on lab

- **Exercise: Design and implement a brightness monitoring and control system using Zephyr RTOS.**
- **15% of the final assessment!**

- **Considerations:**
  - Consider the RGB LED configuration (common anode or common cathode) when implementing its control.
  - The USER buttons in the Nucleo STM32 WL55JC already integrate an integrated debouncing circuit (can you identify its components?).

Hands-on lab

Minimal features:

- Two **operating modes**: NORMAL and BLUE. Consider implementing a Finite State Machine.
- A **short press** of the **B2 button switches** between these modes.
- Event-driven synchronization mechanisms are encouraged (interrupts…).
- In NORMAL mode, brightness must be measured **every 2 seconds** using the ADC. The RGB LED indicates the measured brightness level:
  - **RED**: brightness < 33% / **YELLOW**: 33% ≤ brightness ≤ 66% / **GREEN**: brightness > 66%.
  - Brightness percentages must be calculated with respect to the **full-scale measurement range**.
- The RGB LED may be controlled either using digital or bus outputs.
- In BLUE mode, the RGB LED remains BLUE and ADC acquisition is suspended until NORMAL mode is restored.
- The serial terminal must display the current operating mode and relevant system information, including ADC readings in NORMAL mode.
- Additional **threads beyond the main thread are mandatory**. Each additional thread must be implemented in a separate source file. Students are free to decide how many threads to use, but their design choices must be properly justified. Thread synchronization and communication mechanisms must be appropriately selected and justified.
- **Git** usage is **mandatory**. Students must maintain a repository with regular, meaningful commits throughout development and share access with the professors.
- The repository must include all **source files**, **Zephyr configuration files**, and a **README** explaining the implementation, thread architecture, and build instructions.

Hands-on lab

Extra features:

- A **long press** of the **B2 button** (≥ 1 second) activates OFF mode from either NORMAL or BLUE mode. In OFF mode, the RGB LED is switched OFF, ADC acquisition is suspended, and the message "System OFF" is printed on the serial terminal.
- The long-press event must be **triggered immediately** when the 1-second threshold is reached, without waiting for the button to be released.
- A long press must not trigger a short-press event when the button is released.
- A short press in OFF mode **restores** NORMAL mode.

Assessment:

- NORMAL and BLUE modes, ADC acquisition and RGB LED control: 3.0/10.0
- Thread architecture, synchronization and justification: 2.0/10.0
- Serial terminal output: 1.0/10.0
- Git repository, commit history and README: 1.0/10.0
- OFF mode and long-press detection: 3.0/10.0